Technical Architecture and Methodology

Master’s Thesis: Navegación Autónoma Híbrida en BEV con Optimización de Rutas para Seguimiento de Cultivos

---

1. System Overview

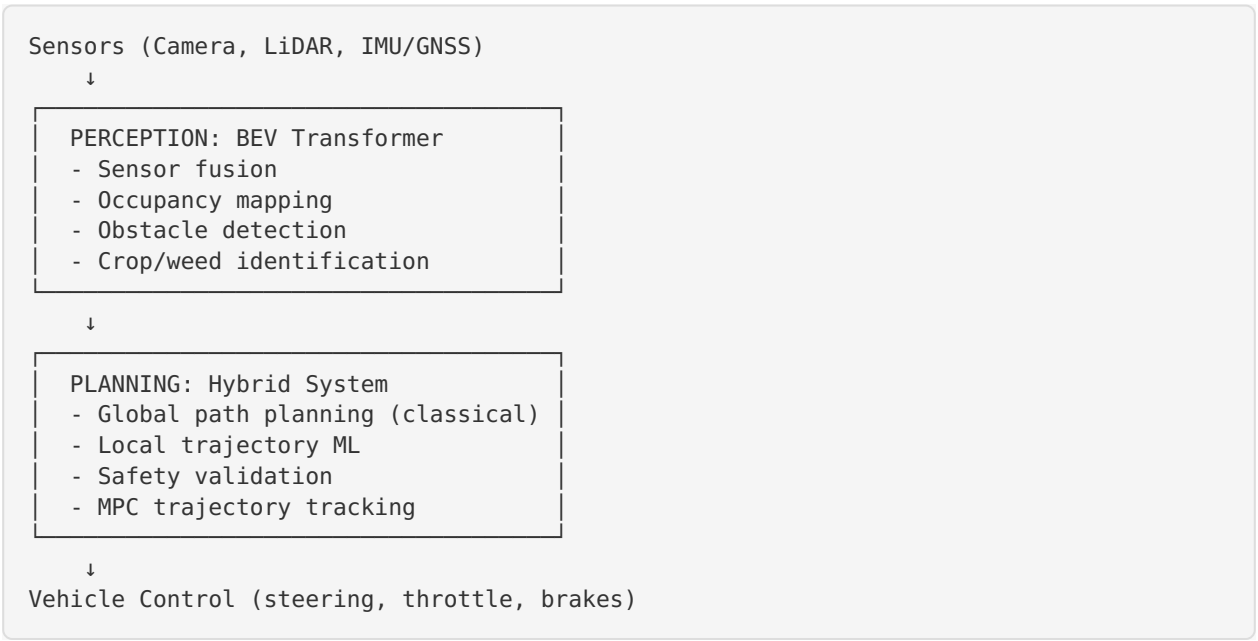
1.1 Architecture Philosophy

The proposed system follows a **modular architecture** with three main coordinated components:

1. **Perception Module**: Multimodal BEV Transformer for environmental understanding
2. **Planning Module**: Hybrid planning system (ML + Classical + MPC)
3. **Identification Module**: Agricultural computer vision for crop/disease/weed classification

This modular design ensures:
- **Safety**: Isolated failure domains
- **Traceability**: Clear error attribution
- **Maintainability**: Independent module updates
- **Robustness**: Classical fallbacks for critical safety functions

1.2 System Pipeline



---

2. Component 1: Multimodal BEV Transformer for Perception

2.1 Objective

Transform heterogeneous sensor inputs into a unified Bird’s-Eye View representation suitable for:
- Spatial reasoning
- Obstacle detection
- Free-space estimation
- Crop row identification
- Path planning input

2.2 Architecture Design

2.2.1 Input Sensors

- **Cameras**: 2-4 RGB cameras (front, rear, sides)
- Resolution: 1280×720 minimum
- FPS: 10-30 Hz
- Field of view: 120°-180°
- **LiDAR**: Single 3D LiDAR
- Range: 30-100m
- Points/second: 300k-1M
- Vertical FOV: 30°-40°
- **IMU/GNSS**: Ego-motion estimation
- IMU: 100-200 Hz
- GNSS: RTK-enabled for cm-level accuracy
- Wheel odometry integration

2.2.2 BEV Transformer Architecture

Encoder Stage:

```
Camera Branch:
  ResNet-50/EfficientNet backbone → Feature extraction (C×H×W)
  ↓
  View transformation (IPM/Lift-Splat) → 3D features
  ↓
  Spatial cross-attention → BEV features (C×H_bev×W_bev)

LiDAR Branch:
  Voxelization/Pillarization → Sparse 3D representation
  ↓
  PointPillars/VoxelNet encoder → 3D features
  ↓
  BEV projection → BEV features (C×H_bev×W_bev)

Fusion:
  Cross-modal attention between camera and LiDAR BEV features
  ↓
  Temporal attention (optional) for motion consistency
```

Decoder Stage:

```
BEV Features → Multiple task heads:
  1. Occupancy Grid Head: Binary occupancy map (200×200 grid, 0.5m resolution)
  2. Semantic Segmentation Head:
     - Free space
     - Obstacles (trees, animals, machinery)
     - Crops (by type)
     - Weeds
     - Traversable/non-traversable terrain
  3. Object Detection Head: 3D bounding boxes for dynamic objects
  4. Uncertainty Estimation Head: Confidence scores
```

2.2.3 BEV Space Configuration

- **Grid size**: 200×200 cells
- **Resolution**: 0.5m per cell (100m×100m coverage)
- **Coordinate frame**: Vehicle-centered, forward-facing
- **Update rate**: 5-10 Hz

2.3 Model Selection

Proposed Architecture: BEVFormer-inspired with agricultural adaptations

Rationale:
- Proven performance on nuScenes, Waymo datasets
- Strong temporal modeling capability
- Efficient multi-scale feature fusion
- Adaptable to agricultural scenarios

Modifications for Agriculture:
- Enhanced ground plane segmentation
- Crop row detection head
- Vegetation height estimation
- Occlusion handling for tall crops

2.4 Training Strategy

2.4.1 Supervised Learning Approach

- **Primary method**: Fully supervised learning with labeled BEV maps
- **Loss functions**:
- Occupancy: Binary cross-entropy + Focal loss
- Segmentation: Cross-entropy + Dice loss
- Detection: Smooth L1 + IoU loss
- Temporal consistency: L2 between consecutive frames

2.4.2 Data Requirements

- **Synthetic data generation**:
- CARLA/LGSVL simulator with agricultural environments
- Unity-based farm simulator
- Domain randomization (weather, lighting, crop types)
- **Real-world data** (if available):

- Annotated agricultural field traversals
- Manual labeling or semi-automatic annotation
- Target: 10k-50k labeled frames

2.4.3 Transfer Learning

- Pre-train on automotive datasets (nuScenes, KITTI)
- Fine-tune on agricultural data
- Progressive unfreezing strategy

2.5 Evaluation Metrics

- **Occupancy accuracy**: Precision, Recall, F1-score, IoU
- **Segmentation**: mIoU (mean Intersection over Union) per class
- **Detection**: mAP (mean Average Precision) at various IoU thresholds
- **Inference speed**: FPS, latency
- **Robustness**: Performance under varying weather, lighting conditions

---

3. Component 2: Hybrid Path Planning System

3.1 Objective

Generate safe, feasible, and efficient trajectories by combining:
- Machine learning for human-like trajectory proposals
- Classical planners for safety and feasibility guarantees
- MPC for smooth trajectory execution under constraints

3.2 Three-Layer Architecture

3.2.1 Layer 1: Global Path Planner (Classical)

Purpose: High-level route planning for field coverage

Algorithms:
- **Coverage path planning**: Boustrophedon decomposition
- **Waypoint optimization**: Traveling salesman variants
- **Area coverage**: Spiral, zigzag, or custom patterns

Inputs:
- Field boundaries (GPS coordinates or map)
- Obstacle map from BEV perception
- Vehicle dimensions, turning radius
- Task requirements (full coverage, partial coverage, specific zones)

Outputs:
- Sequence of global waypoints
- Estimated coverage time
- Fuel/energy consumption estimate

Implementation:
- Python: NetworkX for graph-based planning
- C++: OMPL (Open Motion Planning Library) for sampling-based methods

3.2.2 Layer 2: Local Trajectory Planner (Hybrid ML + Classical)

Purpose: Real-time obstacle avoidance and trajectory generation

ML Component: Trajectory Scoring Network

```
Architecture:
  Input:
    - Current BEV occupancy map (200×200×C)
    - Ego state (position, velocity, heading)
    - Global waypoint goal
    - Candidate trajectories (N×T×3: x,y,θ over T timesteps)

  Encoder:
    CNN for BEV feature extraction
    ↓
    MLP for ego state encoding
    ↓
    Concatenation + Cross-attention

  Trajectory Scoring:
    For each candidate trajectory:
      - Embed trajectory as sequence (LSTM/Transformer)
      - Compute collision probability
      - Compute goal-reaching score
      - Compute smoothness score
      - Output: weighted combined score

  Output: Score for each of N candidate trajectories
```

Learning Method:
- **Imitation Learning** from expert demonstrations
- Collect human driving data in simulation
- Behavior cloning: supervised learning (state → trajectory)
- DAgger for distribution shift mitigation

- **Supervised Learning** with synthetic labels
- Generate safe/unsafe trajectory pairs
- Train binary classifier or regression scorer

Classical Component: Lattice Planner

```
1. Generate trajectory lattice:
   - Discrete set of motion primitives
   - Curvature-continuous paths
   - Respect kinematic constraints (turning radius, velocity limits)

2. Collision checking:
   - Ray-casting against BEV occupancy
   - Safety margin expansion
   - Dynamic obstacle prediction

3. Cost evaluation:
   - Distance to goal
   - Heading alignment
   - Smoothness (curvature, acceleration)
   - Terrain traversability

4. Best trajectory selection:
   - Combine ML scores + classical costs
   - Select top-k safe trajectories
   - Pass to MPC layer
```

Hybrid Decision Logic:

```python
def select_trajectory(candidates, bev_map, ego_state, goal):
    # ML scoring
    ml_scores = trajectory_scorer_net(candidates, bev_map, ego_state, goal)

    # Classical validation
    safe_trajectories = []
    for traj, score in zip(candidates, ml_scores):
        if is_collision_free(traj, bev_map) and \
           is_kinematically_feasible(traj, vehicle_params):
            classical_cost = compute_classical_cost(traj, goal)
            combined_score = alpha * score + (1 - alpha) * (1 / classical_cost)
            safe_trajectories.append((traj, combined_score))

    # Select best safe trajectory
    best_trajectory = max(safe_trajectories, key=lambda x: x[1])[0]
    return best_trajectory
```

3.2.3 Layer 3: MPC Trajectory Tracking (Control)

Purpose: Execute selected trajectory while respecting real-time constraints

MPC Formulation:

```
State: x = [x_pos, y_pos, heading, velocity]ᵀ
Control: u = [steering_angle, acceleration]ᵀ

Objective (at each timestep):
  min  Σ ||x_k - x_ref,k||²_Q + ||u_k||²_R + ||Δu_k||²_S
  u    k=0 to H-1

Subject to:
  x_{k+1} = f(x_k, u_k)           # Vehicle dynamics (bicycle/kinematic model)
  u_min ≤ u_k ≤ u_max              # Actuator limits
  δ_min ≤ δ_k ≤ δ_max              # Steering angle limits
  a_min ≤ a_k ≤ a_max              # Acceleration limits
  x_k ∈ X_safe                     # Safety constraints (no collision)
  κ(x_k) ≤ κ_max                   # Curvature limit

Where:
  H: prediction horizon (10-20 steps)
  Q: state tracking weight matrix
  R: control effort weight matrix
  S: control smoothness weight matrix
```

Vehicle Model:
- **Kinematic bicycle model** for low-speed agricultural operations

`ẋ = v * cos(θ)`
`ẏ = v * sin(θ)`
`θ̇ = (v / L) * tan(δ)`
`v̇ = a`

Where: L = wheelbase, δ = steering angle, a = acceleration

Implementation:
- **Solver**: CasADi or ACADOS for nonlinear MPC
- **Update rate**: 20-50 Hz
- **Horizon**: 2-3 seconds

Safety Layer:
- Emergency brake trigger if no safe trajectory exists
- Geofence boundary enforcement
- Maximum velocity limits based on terrain type

3.3 Training Data Generation

For ML Trajectory Scorer:

1. **Simulation-based expert demonstrations**:
   - Human teleoperation in agricultural simulator
   - Record (BEV, ego_state, goal) → expert trajectory
   - 10k-50k trajectory samples

2. **Synthetic trajectory labeling**:
   - Generate random trajectory candidates
   - Label as safe/unsafe using ground-truth maps
   - Train classifier or scorer

3. **Domain randomization**:
   - Vary field layouts, obstacle configurations

- Different crop types, field conditions
- Weather variations (sunny, rainy, dusty)

3.4 Integration with BEV Perception

```
BEV Perception Output (10 Hz)
    ↓
[Occupancy map + semantic labels]
    ↓
Global Planner (1 Hz) ← Field map
    ↓
[Waypoint sequence]
    ↓
Local Planner (5-10 Hz) ← BEV map + ego state
    ↓
[Selected trajectory]
    ↓
MPC Controller (20-50 Hz)
    ↓
[Control commands: steering, throttle]
```

3.5 Evaluation Metrics

Global Planning:
- Coverage completeness (% of field covered)
- Path length efficiency
- Number of turns/reversals
- Energy consumption estimate

Local Planning:
- Collision-free rate (% of safe trajectories)
- Goal-reaching success rate
- Trajectory smoothness (jerk, curvature variance)
- Computational latency

MPC Tracking:
- Tracking error (RMSE from reference)
- Control smoothness (steering/throttle rate)
- Constraint satisfaction rate

---

4. Component 3: Agricultural Computer Vision - Crop/Disease/Weed Identification

4.1 Objective

Identify and classify:
1. **Crops**: Crop type and health status
2. **Diseases**: Common plant diseases (leaf spot, blight, rust, etc.)
3. **Weeds**: Weed species and density

4.2 Model Architecture

4.2.1 Semantic Segmentation Approach

Architecture: DeepLabV3+ with EfficientNet-B4 backbone

Rationale:
- State-of-the-art segmentation performance
- Atrous convolution for multi-scale context
- Efficient encoder-decoder structure
- Proven on agricultural datasets

Network Design:

```
Input: RGB image (512×512×3)
  ↓
Encoder (EfficientNet-B4):
  - Pretrained on ImageNet
  - Extract multi-scale features
  ↓
ASPP (Atrous Spatial Pyramid Pooling):
  - Capture context at multiple scales
  - Rates: [6, 12, 18] for agricultural objects
  ↓
Decoder:
  - Upsample to original resolution
  - Skip connections from encoder
  ↓
Output: Segmentation mask (512×512×N_classes)
  Classes: Background, Crop-1, Crop-2, ..., Weed-1, Weed-2, ..., Diseased-area
```

4.2.2 Alternative: Two-Stage Approach

Stage 1: Crop/Weed Segmentation
- Segment crop rows vs. weeds vs. soil
- Binary or multi-class segmentation
- Real-time capable (20-30 FPS)

Stage 2: Disease Classification
- Crop patches from Stage 1
- EfficientNet/ResNet classifier
- Multi-label classification for diseases

4.3 Dataset Strategy

4.3.1 User-Provided Dataset

- **Description**: Image dataset for crops, diseases, weeds
- **Required preprocessing**:
- Quality check (resolution, blur, lighting)
- Annotation verification
- Train/val/test split (70/15/15)
- Data augmentation strategy

4.3.2 Augmentation Strategy

```
Training Augmentations:
  - RandomHorizontalFlip(p=0.5)
  - RandomRotation(degrees=15)
  - ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)
  - RandomResizedCrop(scale=(0.8, 1.0))
  - GaussianBlur(kernel_size=5, p=0.3)
  - RandomErasing(p=0.2)  # Simulate occlusion

Agricultural-specific:
  - Simulate different lighting conditions (dawn, noon, dusk)
  - Add synthetic shadows (crop shading effects)
  - Dust/fog overlay (field conditions)
```

4.3.3 Public Datasets for Transfer Learning

- **PlantVillage**: 54k images, 38 crop-disease classes
- **DeepWeeds**: 17k weed images, 8 classes
- **Agriculture-Vision**: 94k images, 6 field anomaly classes
- **Crop/Weed Field Image Dataset (CWFID)**: Labeled crop rows and weeds

Transfer Learning Strategy:
1. Pretrain on ImageNet (general features)
2. Fine-tune on public agricultural datasets (domain adaptation)
3. Final fine-tuning on user's dataset (task-specific)

4.4 Training Strategy

4.4.1 Loss Function

```
Total Loss = w1 * CrossEntropyLoss + w2 * DiceLoss + w3 * FocalLoss

Where:
  - CrossEntropyLoss: Standard segmentation loss
  - DiceLoss: Handle class imbalance (small diseased regions)
  - FocalLoss: Focus on hard examples (rare weed species)

Weights: w1=1.0, w2=0.5, w3=0.3 (tunable)
```

4.4.2 Class Balancing

- **Challenge**: Imbalanced classes (e.g., few disease samples)
- **Solutions**:
- Weighted sampling during training
- Class-weighted loss
- Oversampling minority classes
- Synthetic minority oversampling (SMOTE for images)

4.4.3 Training Schedule

```
Optimizer: AdamW
Learning Rate: 1e-4 with cosine annealing
Batch Size: 16-32 (depending on GPU memory)
Epochs: 100-150 with early stopping
  - Patience: 20 epochs
  - Monitor: Validation mIoU

Learning Rate Schedule:
  - Warmup: 5 epochs linear increase
  - Cosine decay: to 1e-6 over remaining epochs
```

4.5 Integration with BEV System

4.5.1 Spatial Mapping

```
Camera Images → Crop/Disease/Weed Segmentation
    ↓
Project segmentation masks to BEV space using:
  - Camera intrinsics/extrinsics
  - Inverse Perspective Mapping (IPM)
  - Depth information from BEV transformer
    ↓
Fused BEV Semantic Map:
  - Combines geometric occupancy with semantic labels
  - Enables crop-aware path planning
  - Prioritizes areas needing inspection/treatment
```

4.5.2 Task-Specific Path Planning

- **Healthy crop areas**: Standard coverage patterns
- **Diseased areas**: Slower speed, detailed inspection, targeted spraying
- **Weed-heavy areas**: Precise treatment path, minimize crop damage
- **Crop rows**: Follow crop lines, avoid trampling

4.6 Evaluation Metrics

Segmentation Performance:
- **mIoU** (mean Intersection over Union): Primary metric
- **Per-class IoU**: Identify weak classes
- **Pixel Accuracy**: Overall correctness
- **Precision/Recall** per class
- **F1-Score** per class

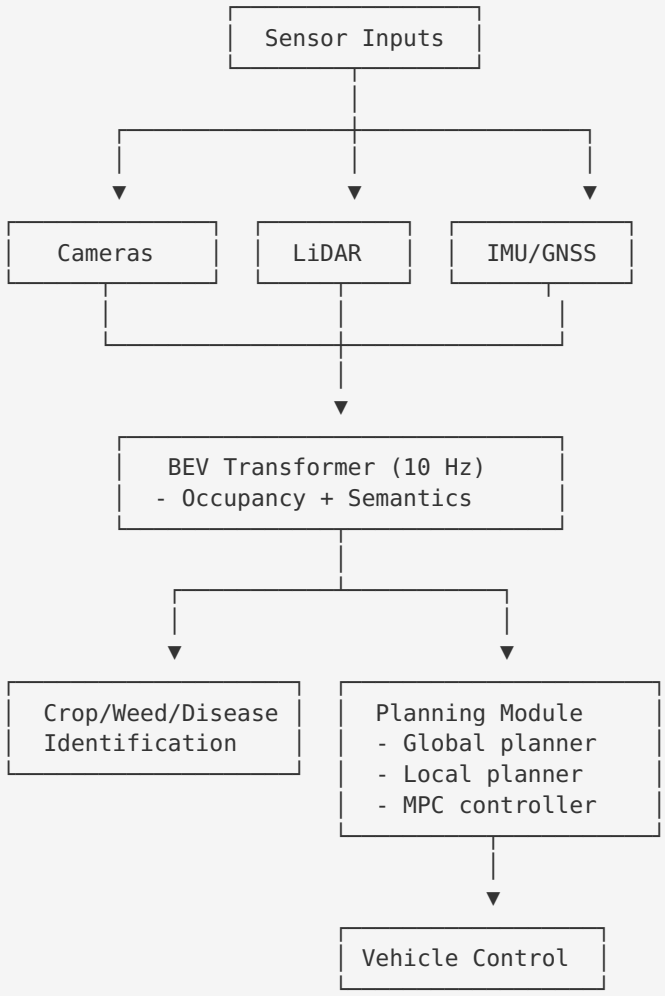
Classification Performance (if two-stage):
- **Accuracy**: Overall classification rate
- **Precision/Recall/F1** per disease type
- **Confusion Matrix**: Identify misclassifications

Real-World Performance:
- **Inference speed**: FPS on target hardware
- **Robustness**: Performance across lighting/weather conditions
- **Generalization**: Performance on unseen fields/crop varieties

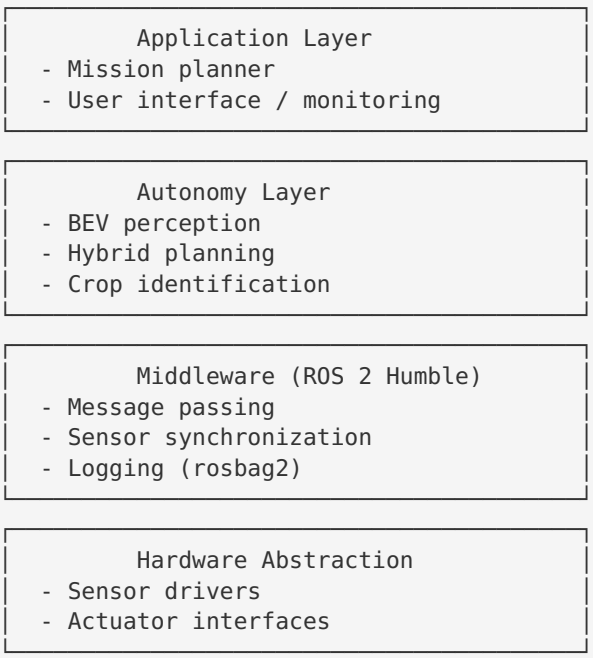
---

5. System Integration

5.1 Data Flow



5.2 Software Stack



5.3 Technology Stack

Deep Learning:
- **Framework**: PyTorch 2.0+
- **Acceleration**: CUDA 11.8+, TensorRT for deployment
- **Libraries**: torchvision, timm, segmentation-models-pytorch

Classical Planning:
- **Language**: C++17 / Python 3.9+
- **Libraries**: OMPL, FCL (collision checking), Eigen (linear algebra)

Optimization (MPC):
- **Solver**: CasADi, ACADOS, or qpOASES
- **Language**: C++ for real-time, Python for prototyping

Simulation:
- **Environments**: CARLA, LGSVL, or Unity-based agricultural sim
- **Physics**: Realistic vehicle dynamics, sensor models

Deployment:
- **OS**: Ubuntu 22.04
- **Containerization**: Docker for reproducibility
- **Hardware target**: NVIDIA Jetson AGX Orin or similar edge device

---

6. Experimental Validation Plan

6.1 Simulation Experiments

6.1.1 BEV Perception Validation

- **Metrics**: Occupancy IoU, object detection mAP, segmentation mIoU
- **Scenarios**:
- Varying lighting (dawn, noon, dusk, cloudy)
- Different crop types and growth stages
- Occluded obstacles (behind vegetation)
- Sensor degradation (dusty camera, partial LiDAR failure)

6.1.2 Planning Validation

- **Metrics**: Coverage %, path efficiency, collision rate, tracking RMSE
- **Scenarios**:
- Simple rectangular field (baseline)
- Irregular field boundary with obstacles
- Dynamic obstacles (animals, other vehicles)
- Narrow passages, tight turns

6.1.3 Crop Identification Validation

- **Metrics**: mIoU, F1-score per class, inference time
- **Test sets**:
- User’s held-out test set
- Public dataset benchmarks
- Synthetic challenging cases (heavy occlusion, poor lighting)

6.2 Ablation Studies

6.2.1 BEV Perception Ablations

- Camera-only vs. LiDAR-only vs. fusion
- With/without temporal attention
- Effect of BEV grid resolution

6.2.2 Planning Ablations

- ML-only vs. Classical-only vs. Hybrid
- Impact of MPC horizon length
- Effect of replanning frequency

6.2.3 Identification Ablations

- Different backbone networks (ResNet, EfficientNet, ViT)
- Single-stage vs. two-stage approach
- Impact of transfer learning vs. training from scratch

6.3 Real-World Testing (if possible)

6.3.1 Data Collection

- Record sensor data (camera, LiDAR, GNSS) in agricultural fields
- Synchronize and calibrate sensors
- Annotate for ground truth

6.3.2 Offline Evaluation

- Run trained models on collected data
- Measure performance in real conditions
- Identify failure modes

6.3.3 Online Testing (if available)

- Deploy on test vehicle
- Supervised autonomous operation
- Safety driver always present
- Incremental complexity (simple field → complex field)

---

7. Implementation Timeline (6 months)

Month 1-2: Foundation

- Literature review completion
- Dataset preparation and augmentation
- Simulation environment setup
- Baseline model implementation (simple versions)

Month 3-4: Core Development

- Full BEV transformer implementation and training
- Hybrid planning system integration
- Crop/disease/weed model training
- Initial integration testing

Month 5: Optimization and Validation

- Model optimization (pruning, quantization if needed)
- Comprehensive simulation experiments
- Ablation studies
- Performance benchmarking

Month 6: Thesis Writing

- Results analysis
- Thesis manuscript drafting
- Presentation preparation
- Code documentation and release

---

8. Expected Contributions

8.1 Technical Contributions

1. **Multimodal BEV architecture** adapted for agricultural environments
2. **Hybrid planning framework** combining ML and classical methods with safety guarantees
3. **Integrated perception-planning-control pipeline** for autonomous agricultural machinery
4. **Benchmark results** on agricultural computer vision tasks

8.2 Datasets and Code

- Annotated agricultural field dataset (if created)
- Open-source implementation on GitHub
- Pre-trained model weights
- Simulation configurations

8.3 Future Work Directions

1. **Pose estimation for implement docking**: 6D pose estimation for autonomous attachment
2. **Multi-agent coordination**: Fleet management for multiple autonomous vehicles
3. **Energy-optimal planning**: Battery/fuel-aware path optimization
4. **Adversarial robustness**: Testing against rare edge cases
5. **Sim-to-real transfer**: Domain adaptation techniques

---

9. Risk Mitigation

9.1 Technical Risks

Risk	Mitigation
Insufficient labeled data	Transfer learning, data augmentation, synthetic data generation
Poor sim-to-real transfer	Domain randomization, fine-tuning on real data if available
High computational cost	Model compression (pruning, quantization), efficient architectures
Sensor failure/degradation	Multi-sensor fusion with fallback modes, uncertainty quantification
Planning infeasibility	Classical safety layer always active, emergency stop logic
Weather/lighting variability	Data augmentation, robust training, test-time adaptation

9.2 Timeline Risks

Risk	Mitigation
Training takes longer than expected	Start with smaller models, parallelize experiments
Simulation environment issues	Use well-established tools (CARLA), have backup options
Integration challenges	Modular design, incremental integration, unit tests
Thesis writing delays	Weekly writing targets, outline early, iterative drafts

---

10. Success Criteria

10.1 Minimum Viable System (Must-Have)

- ✓ BEV perception functional in simulation (mIoU > 0.6)
- ✓ Hybrid planner generates collision-free paths (>95% success)
- ✓ MPC tracks trajectories with low error (RMSE < 0.5m)
- ✓ Crop/weed identification (mIoU > 0.7 on test set)
- ✓ Integrated system runs end-to-end in simulation
- ✓ Complete thesis with methodology, experiments, results

10.2 Target Performance (Should-Have)

- ✓ BEV perception (mIoU > 0.7)
- ✓ Planning collision-free rate > 98%
- ✓ Tracking error < 0.3m
- ✓ Crop identification mIoU > 0.75
- ✓ Real-time capable (>5 Hz full pipeline)
- ✓ Ablation studies completed
- ✓ Comparison with baseline methods

10.3 Stretch Goals (Nice-to-Have)

- ✓ Real-world data collection and testing
- ✓ Open-source code release
- ✓ Paper submission to conference/journal
- ✓ BEV perception mIoU > 0.8
- ✓ Identification mIoU > 0.8

---

11. Conclusion

This technical architecture provides a comprehensive, modular, and safety-conscious design for autonomous agricultural machinery. By combining state-of-the-art deep learning (BEV transformers, semantic segmentation) with proven classical methods (lattice planning, MPC), the system achieves both performance and reliability.

The phased implementation plan, with clear milestones and risk mitigation strategies, ensures the thesis can be completed within the 6-month timeline while producing significant technical contributions to the field of agricultural robotics and autonomous systems.

Document Version: 1.0
Last Updated: 2026-08-19
Author: Master’s Thesis Project
Title: Navegación Autónoma Híbrida en BEV con Optimización de Rutas para Seguimiento de Cultivos